

FlyRank Week 2 - Frame It as Cases

Achyuta Biswal

July 2026

FlyRank Week 2 - Frame It as Cases

Voice Card

Direct, warm, plain, specific, no buzzwords.

Audience

Backend Engineering Manager hiring an entry-level Java Backend Developer.

Desired Action

Shortlist me for a technical interview.

Case Study 1: Employee Task Management System

Problem

In most organizations, task assignment and tracking is still done through spreadsheets, emails, or messaging apps. This makes it hard to monitor progress, know who is responsible for what, and keep any real accountability. There's also no built-in way to control who can see or change what — anyone with access to the spreadsheet or chat thread can technically edit anything.

What I Did (and Why)

I built a backend REST API using Java 23 and Spring Boot 3.2.5 that lets Admins create employees, assign tasks, and manage the system, while Employees can only view and update the status of tasks assigned to them — nothing else. I enforced this separation using Spring Security with role-based authorization, so access control isn't just a UI restriction, it's built into the API layer itself.

For authentication, I used JWT instead of session-based login. REST APIs are stateless by nature, and I wanted to actually understand how token-based auth works in practice rather than relying on sessions.

On the database side, I used MySQL with two main tables — users (credentials and roles) and tasks (title, description, status, due date, assigned employee). Instead of using Hibernate, I deliberately chose Spring JDBC (JdbcTemplate) so I'd have to write the SQL myself and actually understand what was happening at the database level instead of letting an ORM abstract it away.

I organized the API endpoints by responsibility — authentication endpoints, Admin endpoints, and Employee endpoints — so the codebase stays easy to navigate as it grows. I used HikariCP for connection pooling, Maven to build, and Postman to test every endpoint manually.

The hardest part was getting JWT authentication and the Spring Security filter chain working correctly. I didn't understand the full flow at first — how tokens get generated, validated, and used to gate access per role. I broke it down into smaller pieces, went through the official Spring Security docs, compared a few different tutorial implementations, and used Postman plus application logs to trace exactly where requests were failing. I also hit database connection issues with MySQL and HikariCP early on that stopped the app from starting until I fixed the config. Both problems got solved the same way: isolate the failure, check the logs, make one change, test again.

Outcome

The system is fully functional end-to-end — login, JWT-based auth, role-based access for Admins and Employees, employee management, and task assignment/updates, all tested through Postman across roughly 12-15 endpoints. It's on my GitHub, though I haven't deployed it since the goal was to actually understand how to build a secure backend, not to host a demo.

This project showed me that backend development isn't just writing CRUD endpoints — it's thinking about who should be able to touch what data, how requests flow through the system, and how to structure code into layers (controller, service, repository, security) so it doesn't fall apart as it grows. It also changed how I debug: instead of hunting for a quick fix, I got comfortable reading logs and testing step by step until I found the actual root cause.

Case Study 2: Railway Reservation System

Problem

Manual or poorly organized train booking processes lead to double bookings, long queues, incorrect passenger records, and no reliable way to check seat availability. I imagined a system where passengers could search trains, book seats, cancel reservations, and view their booking details in one place — keeping reservation data consistent and reducing errors, without relying on scattered manual processes.

What I Did (and Why)

I built this as a Java console application using Core Java and object-oriented programming — no Spring Boot, no database, no REST layer. I made that choice deliberately: I wanted a project that forced me to focus purely on Java, OOP, and data structures before adding the complexity of a database or web framework.

The design centers on three core classes — Train, Passenger, and Booking — backed by in-memory Java Collections (`List<Train>`, `List<Passenger>`, `List<Booking>`). Each Booking object holds direct references to a Passenger object and a Train object, rather than foreign keys, since there's no database involved. A Main class handles user interaction through a Scanner-based, menu-driven interface.

The core logic is the booking workflow: when a user requests a booking, the system finds the selected Train, checks its available seat count, creates a Booking linking the passenger and train, adds it to the booking list, and decrements the train's available seats. Cancelling a reservation removes the booking and increments the seat count back. This keeps seat availability accurate and prevents overbooking within a single running session.

The main challenge was getting this booking and cancellation logic right — making sure every booking correctly linked a passenger and train, and that available seats stayed accurate through repeated bookings and cancellations. I didn't build in any concurrency handling, like preventing two bookings for the same seat at the same instant, since the project's scope was the core reservation logic. I tested it by manually running the console app through different scenarios — booking, cancelling, booking when no seats were left — and checking that seat counts and booking records stayed consistent every time.

Outcome

The result is a fully working console application: users can view trains, book tickets, cancel reservations, and view booking details through a menu, with seat availability updating correctly across every scenario I tested by hand. Since it's in-memory, data resets when the program closes — an accepted tradeoff for a project focused on Java fundamentals rather than persistence.

This project taught me that good design is about organizing responsibilities clearly, not just making something work — splitting the system into Train, Passenger, and Booking classes made the logic much easier to reason about and extend. It also reinforced that business rules, like checking seat availability before confirming a booking, matter as much as the data structures holding the data. Working through this gave me the OOP foundation that made picking up Spring Boot for the Employee Task Management System noticeably easier.

Case Study 3: FlyRank Backend AI Internship Assignments

Problem

Before this internship, my GitHub was mostly finished code with little explanation of the problem it solved or the reasoning behind it. There was no clear way for someone reviewing my work to understand how I think, make decisions, or actually use AI in my development process — the portfolio just showed output, not process.

What I Did (and Why)

I worked through a structured set of assignments to rebuild my portfolio around evidence of my thinking, not just a list of finished projects.

I started with an AI Workflow Audit, setting up ChatGPT and Claude alongside VS Code, Git/GitHub, and Postman, and figuring out where AI actually helps versus where I need to do the work myself. The conclusion I landed on: AI is most useful as a thinking partner — explaining concepts, reviewing code, debugging alongside me, helping me interview myself about my own decisions — not as something that writes or decides for me. I still make the technical calls, verify anything AI generates, and rewrite anything AI drafts so it's in my own voice.

From there I built a Portfolio Sitemap and Toolkit — a plan for the site's structure (Home, About, Projects, Skills, Achievements, Contact) and a documented list of the tools that support my actual workflow, not just a tech buzzword list.

I also changed how I use GitHub and Git day-to-day. Before this, repos were just where I dumped finished code. Now each project has its own folder, a README that explains the purpose, features, stack, and setup instead of one line of description, and I commit in smaller, meaningful increments instead of one big commit at the end. I got more comfortable handling things like push conflicts with `git pull --rebase` along the way.

For documentation, the focus was making my reasoning visible, not producing formal specs — I wrote up the purpose, approach, decisions, and outcome for each assignment, including structured Week 1 submissions, and I'm now applying the same thinking to turning my projects into case studies like this one.

The clearest example of the AI-as-thinking-partner approach in practice: while debugging JWT authentication in my Employee Task Management System, I used AI to explain how the security filter chain and JWT filter fit together, then tested and verified everything myself with Postman and logs rather than just copying a working solution. Same thing with a MySQL/HikariCP connection error — AI helped me interpret the error, but I still tracked down and fixed the actual misconfiguration myself.

Outcome

I now have completed Week 1 submissions (AI Workflow Audit, Portfolio Sitemap and Toolkit, proof statement), reorganized GitHub repos with real documentation, and case studies in progress for my projects that explain reasoning instead of just listing what was built. More importantly, I have a much clearer sense of what I'm actually trying to prove to a hiring manager — not "I know Java and Spring Boot," but that I can design secure systems, explain my decisions, debug through understanding rather than guesswork, and use AI as a tool without letting it replace my own judgment. Going forward, I want to document as I build instead of reconstructing it afterward, since that's clearly made this process easier and more accurate.

Professional Bio

I'm a third-year B.Tech Computer Science and Engineering student at GIET, Baniatangi, Bhubaneswar, under Biju Patnaik University of Technology, expecting to graduate in 2027. I'm building toward a

career as a Java backend developer, with hands-on experience designing REST APIs using Spring Boot, Spring Security, JDBC, and MySQL.

My background includes academic projects like an Employee Task Management System with JWT authentication and role-based access control, and a Railway Reservation System built around core Java and object-oriented design. I also completed a 10-week Java Full Stack Development Internship through EduSkills (AICTE), where I built a full-stack web application using Spring Boot, JPA, and MySQL and applied SDLC principles from design through implementation, strengthening my understanding of REST API development, MVC architecture, and debugging.

Through the FlyRank Backend AI Engineering Internship, I've worked on organizing my projects into case studies that explain not just what I built, but the problems I was solving and the decisions behind them. Together, these experiences show my progression from learning core Java to building secure backend applications and communicating that work clearly.

Contact / Call-to-Action

I'm looking for an entry-level Java Backend Developer role where I can keep building on this foundation — designing backend systems, working through real problems, and learning from engineers with more experience than me. If my projects and the way I think through them are a fit for your team, I'd welcome the chance to talk through my work in a technical interview.

Achyuta Biswal Bhubaneswar, Odisha Email: achyutabiswal977@gmail.com Phone: +91 7846807494 LinkedIn: www.linkedin.com/in/achyuta-biswal GitHub: github.com/AchyutaBiswal

Before vs After: Employee Task Management System

Before (generic AI-style writing):

"Leveraged cutting-edge technologies to build a scalable, robust solution for enterprise task management. As a results-driven developer, I engineered an innovative, industry-leading platform utilizing Spring Boot and JWT to deliver a world-class user experience with seamless role-based access control."

After (my authentic version):

"I built a backend REST API using Java and Spring Boot that lets Admins create employees and assign tasks, while Employees can only view and update the status of tasks assigned to them — nothing else. I used JWT instead of session-based login because REST APIs are stateless, and I wanted to understand how token-based auth actually works. The hardest part was getting the Spring Security filter chain configured correctly — I broke it down into smaller pieces, read the official docs, and used Postman and application logs to trace exactly where requests were failing until I understood why it worked."

Why the difference matters: The “before” version could describe almost any project — it says nothing about what was actually built, what decisions were made, or what problems came up. The “after” version is specific: it names the real technical choice (JWT over sessions), explains the reasoning, and shows the actual debugging process. That specificity is what tells a hiring manager I understand what I built, rather than just being able to describe it in impressive-sounding terms.